Appendix A: Project Proposal

DELIVERABLE #1 • PHASE 1 / CHECKPOINT 1

1. PROJECT & TEAM IDENTIFICATION

ITEM	DETAILS
Group name	Team FindIt (Course / Section: _____)
Project title	**FindIt:** Containerized Campus Lost & Found Management System with Automated Jenkins CI/CD
Short description (3 to 5 sentences)	FindIt is a multi-module web application that enables students and staff to report lost belongings, post found items, search active listings, and submit or review recovery claims. When a reviewer approves a claim, the system automatically transitions the item's status to `returned`. The application is split into three containerized modules—a static Web UI (`frontend`), an Items microservice (`items-api`), and a Claims microservice (`claims-api`)—routed through a single public Nginx reverse proxy (`proxy`) and backed by a persistent PostgreSQL database (`db`). Every push to GitHub automatically triggers a containerized Jenkins pipeline that runs multi-stage Docker unit tests, builds version-tagged images, deploys via Docker Compose, and verifies live health endpoints.
Repository URL	`https://github.com/dgmdev-sql/findit` *(Shared repository with all 5 members as collaborators)*
Project Lead / Scrum Master	**Josh** *(Owns: Project proposal, task board, architecture diagram, presentation flow, contribution log)*
DevOps / CI-CD Engineer	**Kim** *(Owns: Containerized Jenkins setup, root `Jenkinsfile`, credentials, webhook/Poll SCM trigger, rollback)*
Infrastructure Engineer	**Kennedy** *(Owns: Multi-stage `Dockerfile`s, `docker-compose.yml`, Nginx reverse proxy, networks, volumes, health checks)*
Backend and Database Engineer	**Rey** *(Owns: `items-api` and `claims-api` Flask modules, PostgreSQL `db/init.sql` schema, `pytest` unit tests)*
Frontend, QA, and Documentation Lead	**Dom** *(Owns: `frontend` UI module with build badge, smoke test verification, `README.md`, technical documentation PDF, slides)*

2. APPLICATION MODULES & SUPPORTING CONTAINERS

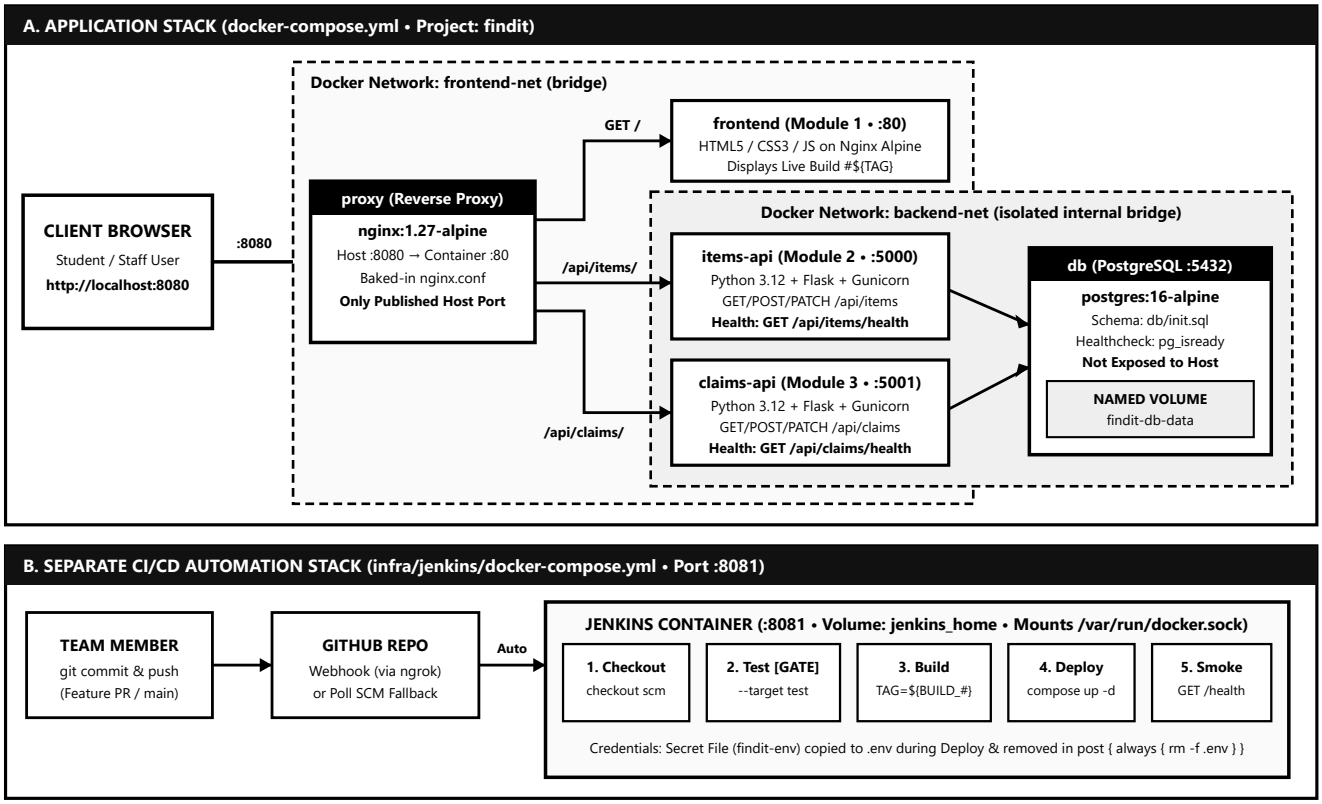
MODULE / CONTAINER	TECHNOLOGY	PURPOSE	JUSTIFICATION
proxy *(Reverse Proxy)*	Nginx 1.27 (`nginx:1.27-alpine`)	Single public entry point (`:8080`); routes `/` to `frontend`, `/api/items/` to `items-api`, and `/api/claims/` to `claims-api`.	Nginx Alpine has a tiny (~40 MB) footprint, decouples public routing from internal container ports, and lets us bake `nginx.conf` into a custom image to avoid Docker-in-Docker bind-mount path errors.
frontend *(App Module 1)*	HTML5, CSS3, ES6 JS on `nginx:1.27-alpine`	User-facing Campus Lost & Found interface for reporting items, filtering listings, submitting claims, and showing the live `Build #`.	Vanilla HTML/CSS/JS served in its own Nginx container builds in under 2 seconds, stays decoupled from backend runtimes, and visibly renders the injected `BUILD_NUMBER` during live demos.
items-api *(App Module 2)*	Python 3.12 + Flask + Gunicorn (`python:3.12-slim`)	REST API managing lost and found item records (`GET`, `POST`, `PATCH /api/items`) and exposing `GET /api/items/health`.	Python 3.12 with Flask and Gunicorn provides a clean, lightweight microservice structure with fast startup times and seamless integration with `pytest` in a multi-stage `Dockerfile`.
claims-api *(App Module 3)*	Python 3.12 + Flask + Gunicorn (`python:3.12-slim`)	REST API managing claim submissions, reviewer approval/rejection, item status transitions to `returned`, and `GET /api/claims/health`.	Separating claim workflows into a dedicated microservice satisfies the 3-module requirement cleanly and isolates claim validation rules and unit tests from catalog browsing.
db *(Database)*	PostgreSQL 16 (`postgres:16-alpine`)	Stores relational `items` and `claims` tables in named volume `findit-db-data`; initialized via `db/init.sql`.	PostgreSQL 16 Alpine enforces relational foreign-key integrity between items and claims, while `pg_isready` provides reliable native container health checks.
jenkins *(Separate Compose)*	Jenkins LTS + Docker CLI (`jenkins/jenkins:lts-jdk17`)	Automated CI/CD server in `infra/jenkins/docker-compose.yml` with persistent volume `jenkins_home`.	Running Jenkins in a separate Compose file with its own persistent volume ensures our application pipeline never restarts or overwrites the CI/CD server itself.

CI/CD Pipeline Plan & Draft Architecture Diagram

3. CI/CD PIPELINE PLAN

PIPELINE PLAN	DETAILS
Git hosting	**GitHub** shared repository (feature branches merged via Pull Requests into `main`).
Trigger (webhook / polling)	**GitHub Webhook** (exposed to local Jenkins via `ngrok http 8081` for instant live-demo triggers), with **Poll SCM (`H/2 * * * *`)** configured as a fallback if campus/lab internet blocks tunnels.
Planned stages	**5 Declarative Stages:** `1. Checkout` → `2. Test (Quality Gate)` → `3. Build Images (TAG=${BUILD_NUMBER})` → `4. Deploy (with Jenkins Credentials)` → `5. Smoke Test / Verify` + `post` cleanup.
Tests to be run	**1. Multi-Stage Unit Tests (`pytest` via `docker build --target test`):** Item creation validation, filtering, missing-field rejection (`400`), claim on nonexistent item (`404`), duplicate claim prevention on returned items (`400`), claim approval status transition, and `/health` checks. **2. Post-Deploy Smoke Tests:** Automated HTTP checks through Nginx proxy to `http://localhost/api/items/health` and `http://localhost/api/claims/health`.
Planned bonus features (optional)	• **Network Segmentation:** Separate `frontend-net` and `backend-net` networks so `proxy` and `frontend` cannot reach PostgreSQL directly. • **Automated Tests on All Backend Modules** & **Build-Tagged Rollbacks (`TAG=${BUILD_NUMBER}`)**.

4. DRAFT SYSTEM ARCHITECTURE & CI/CD FLOW DIAGRAM



STARTUP ORDER & HEALTH CHECK GUARANTEES

1. `db` starts first and runs `pg_isready -U ${POSTGRES_USER} -d findit` every 10s.
2. `items-api` and `claims-api` wait for `db` (`condition: service_healthy`) and expose `/api/*/health`.
3. `proxy` waits for `frontend`, `items-api`, and `claims-api` before accepting traffic on `:8080`.

SECURITY & ISOLATION HIGHLIGHTS

1. **Port Isolation:** Only `proxy` exposes port `8080` to the host; PostgreSQL `:5432` is unreachable from outside Docker.
2. **Network Segmentation:** `proxy` and `frontend` are on `frontend-net` only; `db` is on `backend-net` only.
3. **Non-Root Containers:** Both Python APIs run as an unprivileged `appuser` in production.